



SIMATS

ENGINEERING



Student Name : Devayani P.

Reg no. – 192421082

Course Code - **CSA37**

Course Name - **Software Testing**

Assessment No. – 1

Tool no. –1



SIMATS
ENGINEERING



SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing

COURSE CODE: CSA37

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3

4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3
5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3
9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3
11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3

12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3
15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3
17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3
18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3

20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3
----	--	---

Code of Conduct:

I Devayani P. (192421082) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

Answers :

1. Testing as an Engineering Activity – Banking Application

Testing should not be done randomly. It should be treated as an **engineering activity**, where testing is planned, designed, executed and measured systematically.

For a banking application, the testing team can first identify important functions such as login, fund transfer, balance enquiry and withdrawal. Test cases can then be designed using techniques like **equivalence partitioning, boundary value analysis and decision table testing**.

Testing tools such as **Selenium** for web testing, **JMeter** for performance testing and defect tracking tools like **Jira** can be used. Standards and guidelines such as **IEEE/ISO testing practices** can also be followed to maintain consistency.

Instead of ad-hoc testing, testers should use:

- Requirement-based testing
- Boundary value analysis
- Equivalence partitioning
- Decision table testing
- Regression testing

Example: For fund transfer, test valid and invalid account numbers, zero/negative amounts, insufficient balance and maximum transfer limits.

This systematic approach helps find defects earlier and ensures important functions are properly tested. Hence, it improves the reliability and quality of the banking application.

2. Structured Testing Process for ATM Withdrawal

A structured testing process should be followed throughout the **SDLC** instead of testing only after development.

The main stages are:

1. **Requirement analysis** – Understand rules such as withdrawal limits, PIN validation and insufficient balance.
2. **Test planning** – Decide scope, resources, tools, schedule and risks.
3. **Test case design** – Prepare test cases for valid and invalid withdrawals.
4. **Test execution** – Execute the test cases and record actual results.
5. **Defect reporting** – Report failures clearly with steps, expected result and actual result.
6. **Retesting and regression testing** – Verify fixes and ensure old functions are not affected.

For ATM withdrawal, test cases can include:

- Correct PIN + sufficient balance → cash should be dispensed.
- Wrong PIN → transaction should be rejected.
- Amount greater than balance → transaction should fail.
- Amount above daily limit → transaction should be rejected.

Following the same process for every release reduces missed test cases and inconsistent results. It improves **accuracy, traceability and reliability** of the ATM system.

3. Error, Defect and Failure in Login and Transactions

Error, defect and failure are related but different terms.

- **Error:** A mistake made by a developer or other person.
- **Defect:** A fault introduced into the software because of that error.
- **Failure:** When the software actually behaves incorrectly during execution.

Example: Suppose a developer misunderstands the login requirement and writes code that accepts an incorrect password.

- Developer misunderstanding = **Error**
- Incorrect password validation code = **Defect**

- User successfully logging in with the wrong password = **Failure**

Another example in a transaction system is when the developer uses the wrong calculation formula. The incorrect formula is the defect, and displaying the wrong transaction amount to the customer is the failure.

If testers confuse these terms, defect reports may become unclear. Reports should clearly mention the **defect, steps to reproduce, expected result, actual result, severity and environment**.

Understanding these terms helps developers and testers communicate properly and resolve problems faster.

4. Defect Clustering and Test Prioritization

Defect clustering means that a small number of modules usually contain a large number of defects. Therefore, testing effort should not always be distributed equally.

For example, in an e-commerce application, the payment and checkout modules may contain more defects than the product display module because they have complex business logic and integrations.

Test cases can be prioritized based on:

- Business importance
- Risk
- Previous defect history
- Complexity of the module
- Frequency of use

For example:

Module	Priority
Payment	High
Login	High

Module	Priority
Checkout	High
Product display	Medium
Help page	Low

Defect clustering helps testers focus more effort on risky areas. This improves **defect detection, testing efficiency and time management**.

Real-time example: If previous releases show that payment failures occur frequently, testers should give more regression and negative testing to the payment module in the next release.

5. Defect Classification and Defect Repository

A **defect repository** is a central place where information about defects is stored. It helps the testing team learn from previous mistakes.

Defects can originate due to:

- Incorrect requirements
- Design mistakes
- Coding errors
- Integration problems
- Configuration issues
- Performance problems

They can also be classified as **functional, UI, performance, security, data or integration defects**.

For example, if a banking application repeatedly has incorrect interest calculations, previous defect records can show that similar problems occurred because of calculation logic.

Testers can use repository information to create additional regression test cases for the affected area.

To avoid repeated defects:

- Maintain complete defect records.
- Identify recurring defect patterns.
- Add regression test cases.
- Perform root-cause analysis.
- Share lessons learned with developers.

Thus, a defect repository becomes useful not only for tracking defects but also for **defect prevention and improving future releases**.

6. Mobile Banking Application Crashes During Fund Transfer

Frequent crashes during fund transfer indicate that the application needs systematic testing rather than only normal functional testing.

First, testers should identify the conditions under which the crash occurs. They can test:

- Insufficient balance
- Invalid account number
- Large transfer amount
- Poor network connection
- Session timeout
- Multiple transfers at the same time
- Unexpected app closure

Logs, crash reports and test results should be collected to identify the root cause.

For example, if the application crashes whenever the network is disconnected during a transfer, the problem may be related to poor exception handling.

Systematic testing using **functional, negative, integration, performance and recovery testing** can reveal such problems.

The team should also perform regression testing after fixing the crash. Automated tests can be added for important transfer scenarios.

This approach helps identify the actual cause instead of repeatedly fixing symptoms and makes future fund transfers more reliable.

7. Testing vs Debugging

Testing and debugging are different activities.

Testing is performed to **find failures or defects**, while debugging is performed by developers to **find the cause of a defect and fix it**.

For example, a tester finds that the ATM deducts ₹5,000 but dispenses only ₹2,000. This is testing identifying a failure.

The developer then examines the code, database and transaction logic to find why the wrong amount was processed. This is debugging.

Testing	Debugging
Finds failures	Finds causes
Mainly performed by testers	Mainly performed by developers
Compares expected and actual results	Investigates code and logic
Produces defect reports	Produces fixes

If developers treat debugging as testing, important scenarios may be missed. A proper process should be:

Test → Report defect → Debug → Fix → Retest → Regression test

This separation makes defect resolution more systematic and improves software quality.

8. Cross-Browser Testing for E-Commerce Application

An e-commerce application should be tested on different browsers because the same web page may behave differently depending on the browser, version or operating system.

The testing strategy should cover browsers such as **Chrome, Firefox, Edge and Safari**, along with commonly used mobile browsers.

Testers should check:

- Login
- Product search
- Add to cart
- Payment
- Page layout
- Buttons and links
- JavaScript functionality

Defects may occur because of differences in **HTML/CSS rendering, JavaScript support, browser versions or responsive design**.

Tools such as **Selenium, Playwright and BrowserStack** can help automate and execute tests across multiple browser environments.

For example, the checkout button may work correctly in Chrome but not respond in Safari. Cross-browser testing can identify this before customers face the problem.

Thus, systematic browser testing improves compatibility, user experience and application reliability.

9. Defect Classification in Payroll System

Incorrect salary calculation is mainly a **functional or logical defect** because the system does not implement the required salary calculation correctly.

For example, suppose:

Net Salary = Basic Salary + Allowances – Deductions

If the program accidentally subtracts allowances instead of adding them, the calculation will be wrong.

Defects in a payroll system can include:

- Calculation defects
- Requirement defects
- Data defects
- Database defects
- UI defects
- Integration defects

Understanding defect classes helps testers select suitable test cases.

For calculation defects, testers should use:

- Boundary value analysis
- Equivalence partitioning
- Decision tables
- Data-driven testing

The team should also perform unit testing and regression testing after fixing the calculation.

Proper classification makes it easier to identify common defect patterns and prevent similar salary calculation problems in future releases.

10. Structured Test Case Design Instead of Random Testing

Random testing may find some defects, but it cannot guarantee proper coverage. Testing should be treated as an engineering activity with **planned and systematic test case design**.

First, testers should understand the requirements and identify important functions. Then they should prepare test cases with:

- Test case ID
- Preconditions
- Input data

- Steps
- Expected result
- Actual result
- Pass/Fail status

Useful techniques include:

- Equivalence partitioning
- Boundary value analysis
- Decision tables
- State transition testing
- Use-case testing

For example, if a system accepts an age between 18 and 60, boundary tests should include **17, 18, 19, 59, 60 and 61**.

Structured test design improves coverage and reduces the possibility of missing important conditions. It also makes testing repeatable and easier to maintain.

11. Data Inconsistency in Healthcare Application

Data inconsistency occurs when different modules display different values for the same information.

For example, a patient's blood group may be shown as **O+ in the registration module but A+ in the billing or medical-record module**.

Such defects may originate due to:

- Incorrect database updates
- Duplicate data
- Poor integration
- Incorrect mapping between modules
- Concurrency problems

Testers should perform **integration testing, database testing and data validation**.

Important checks include:

- Same patient ID across modules
- Correct database values
- Correct data transfer between services
- Update and delete consistency
- Transaction rollback

Defects should be recorded with patient/module information, steps, expected value and actual value, while avoiding unnecessary sensitive information.

Systematic testing ensures data remains accurate across modules and reduces the risk of serious healthcare-related errors.

12. Repeated Defects Using a Defect Repository

If the same defects keep appearing, the organization is probably using the defect repository only for tracking and not for learning.

The repository should contain information such as:

- Defect type
- Module
- Severity
- Root cause
- Release
- Resolution
- Repeated occurrence

For example, if login defects appear in three consecutive releases, the team can identify a pattern. Testers can then add stronger login regression tests.

Repository data can also be used to identify:

- High-defect modules
- Common defect types
- Developers/modules needing additional review
- Frequently failing requirements

Root-cause analysis should be performed for repeated defects.

Using repository data in test planning changes it from a simple **record-keeping system into a defect-prevention tool**. This reduces recurring defects and improves release quality.

13. Web Learning System Fails During Peak Usage

The failure during peak usage indicates that the system may not have been tested for realistic workloads.

Normal functional testing may verify that the application works for one or a few users, but it does not prove that it can handle thousands of users.

Testers should perform:

- Load testing
- Stress testing
- Volume testing
- Endurance testing
- Performance monitoring

For example, if an online learning system normally has 500 users but may receive 10,000 users during an examination, testing should simulate that expected load.

Tools such as **JMeter** or **LoadRunner** can be used to generate virtual users.

Test coverage should include login, course access, video streaming, assignment submission and database operations.

Proper performance testing identifies bottlenecks before production and improves system reliability during peak usage.

14. Early Involvement of Testers in SDLC

Testers should not wait until coding is completed. They should participate from the **requirements and design stages**.

During requirement analysis, testers can identify:

- Missing requirements
- Ambiguous statements
- Incorrect business rules
- Unclear acceptance criteria

For example, suppose a banking requirement says, *“Users can transfer money up to the allowed limit.”* A tester can immediately ask what the exact limit is and what should happen when the limit is exceeded.

This prevents misunderstandings from reaching the coding stage.

Testers can also review design documents, prepare test scenarios early and participate in sprint planning.

Early tester involvement reduces the cost of fixing defects because defects found in requirements are generally easier and cheaper to correct than defects found after deployment.

Better collaboration between **developers, testers, business analysts and product owners** improves overall quality.

15. Incomplete Test Coverage

A system may produce correct results for some cases but fail in others because important test conditions were not covered.

For example, a money transfer function may work correctly for ₹1,000 but fail for:

- ₹0

- Negative amounts
- Maximum transfer amount
- Insufficient balance
- Invalid account
- Network failure

Testers should identify different input conditions and use techniques such as **equivalence partitioning and boundary value analysis**.

They can also use:

- Requirement coverage
- Code coverage
- Decision coverage
- Risk-based testing
- Negative testing

A traceability matrix can be used to ensure every important requirement has corresponding test cases.

Better coverage increases the chance of detecting hidden defects and improves confidence that the system will behave correctly in real-world situations.

16. Improving Defect Reporting in Logistics Application

Poor defect reporting delays communication between testers and developers. A defect report should be **clear, complete and reproducible**.

A good defect report should contain:

- Defect ID
- Short title
- Environment

- Preconditions
- Steps to reproduce
- Expected result
- Actual result
- Severity
- Priority
- Screenshots/logs if required

For example:

Title: Delivery status not updated after successful shipment.

Steps:

1. Create an order.
2. Mark it as shipped.
3. Open the tracking page.

Expected: Status should display “Shipped”.

Actual: Status still displays “Processing”.

Clear reporting helps developers reproduce and fix the issue quickly.

The team should use a common defect template and define severity and priority clearly. This improves communication, tracking and turnaround time.

17. Process-Based Testing for Consistency Across Modules

When different modules are tested in different ways, some defects may be missed. To solve this, the team should follow a **standard testing process** for all modules.

The process can include:

1. Understanding requirements.
2. Preparing a test plan.
3. Designing test cases using standard techniques.

4. Executing and recording test results.
5. Reporting and tracking defects.
6. Performing regression testing after fixes.

For example, in a banking application, the **login, fund transfer and account statement modules** should follow the same format for test cases, defect reporting and result recording.

Standard templates, checklists and tools like **Jira** can help maintain consistency. Regular test reviews and common testing guidelines can also be followed.

A standardized process makes testing **repeatable and measurable**. It reduces missed test cases and improves the overall quality of all modules.

18. Edge Cases and Boundary Testing

Edge cases are situations that occur at the limits of normal input. They are often missed when testers mainly test common or normal conditions.

Boundary Value Analysis (BVA) is useful for finding such defects. It tests values at, just below and just above the allowed limits.

For example, if an ATM allows withdrawal from **₹100 to ₹20,000**, test cases should include:

- ₹99 – Invalid
- ₹100 – Valid
- ₹101 – Valid
- ₹19,999 – Valid
- ₹20,000 – Valid
- ₹20,001 – Invalid

Testers should also consider other edge cases such as empty input, maximum file size, expired sessions and maximum transaction limits.

Requirements should be reviewed to identify all boundaries, and boundary test cases should be included in regression testing.

This approach helps detect defects that normal testing may miss and prevents unexpected failures after deployment.

19. Defect Repository and Proper Documentation

A **defect repository** is a centralized place where all software defects are recorded and managed. If defects are not properly documented, the same problems may be repeated and tracking becomes difficult.

Each defect should contain important details such as:

- Defect ID
- Title and description
- Steps to reproduce
- Expected result
- Actual result
- Severity and priority
- Environment
- Status
- Root cause and resolution

For example, if a payment failure occurs repeatedly, the repository can show whether the same defect occurred in earlier releases.

A structured repository also helps testers identify **defect patterns**, such as a particular module having many defects.

Tools such as Jira or other defect-tracking systems can be used. The team should use a common reporting format and regularly review old defects.

Proper documentation improves communication, prevents duplicate work and supports **regression testing and defect prevention**.

20. Redesigning Testing for Banking Transaction Failures

Repeated transaction failures usually indicate that testing is not systematic enough. The testing approach should be redesigned based on requirements, risk and different user conditions.

First, testers should identify important transaction scenarios such as:

- Successful transaction
- Insufficient balance
- Invalid account number
- Invalid transaction amount
- Daily transaction limit
- Network failure
- Duplicate transaction
- Session timeout

Techniques such as **equivalence partitioning, boundary value analysis, decision table testing and negative testing** should be used.

For example, if the daily transfer limit is ₹1,00,000, testers should check amounts below, equal to and above the limit. They should also check what happens if the network fails after the amount is deducted.

After fixing defects, **retesting and regression testing** should be performed. Important transaction cases can also be automated.

Thus, structured testing improves defect detection, reduces repeated failures and gives customers a more reliable banking transaction system.